End-to-End Machine Learning Project

IAI600

Recommended reading

- Chapter 4. Training Models: pages 111-142 including intro to logistic regression

Objective

- Introduce main steps of a ML project based on a simple task
- Provide theoretical guidance to the first three labs

Content

- Identify main objective
- Study the data
- Preprocess the data for ML algorithms
- Select a proper ML model and train it
- Fine-tune the ML model
- Self-study: deployment and maintaining.

Identify main objective

- Build a ML-based solution for predicting house prices based on some input data
- The solution should be at least as good as the existing manual solution

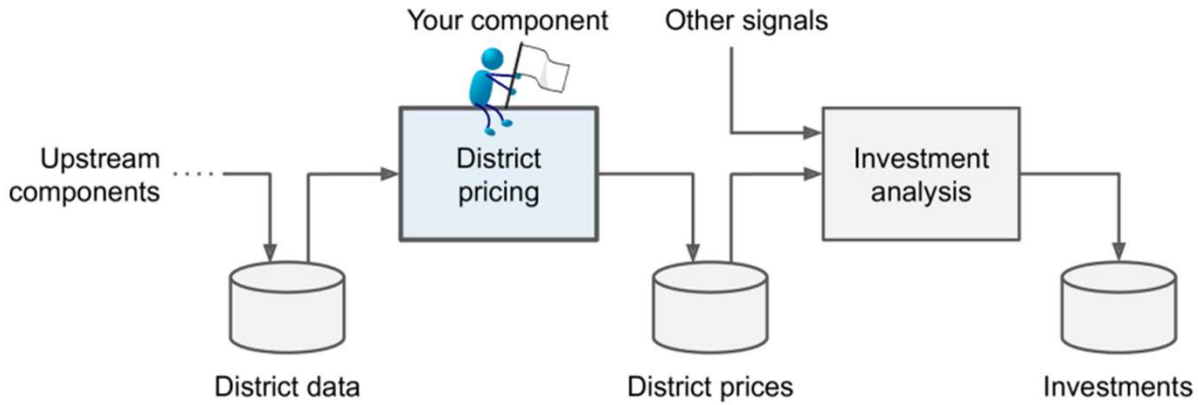


Figure 2-2. A Machine Learning pipeline for real estate investments

Pipeline: A sequence of data processing components

The Data

```
In [6]: housing.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
longitude             20640 non-null float64
latitude              20640 non-null float64
housing_median_age    20640 non-null float64
total_rooms           20640 non-null float64
total_bedrooms        20433 non-null float64
population            20640 non-null float64
households            20640 non-null float64
median_income         20640 non-null float64
median_house_value    20640 non-null float64
ocean_proximity       20640 non-null object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

Figure 2-6. Housing info

Formal Definition of the Problem

- Type of ML problem
  - Supervised learning
    - labeled training examples available
  - Multiple regression problem
    - Multiple features are used to predict a single value
  - Univariate regression
    - A single value is predicted (house value)
  - Batch learning

Select a Performance Measure

- Should tell how close a prediction is to the real value
  - **Distance** between two points

- Two common metrics in ML
  - Root Mean Square Error (RMSE) aka **l2**
  - Mean Absolute Error (MAE) aka **l1**

  - **The lower the better**

RMSE vs MAE

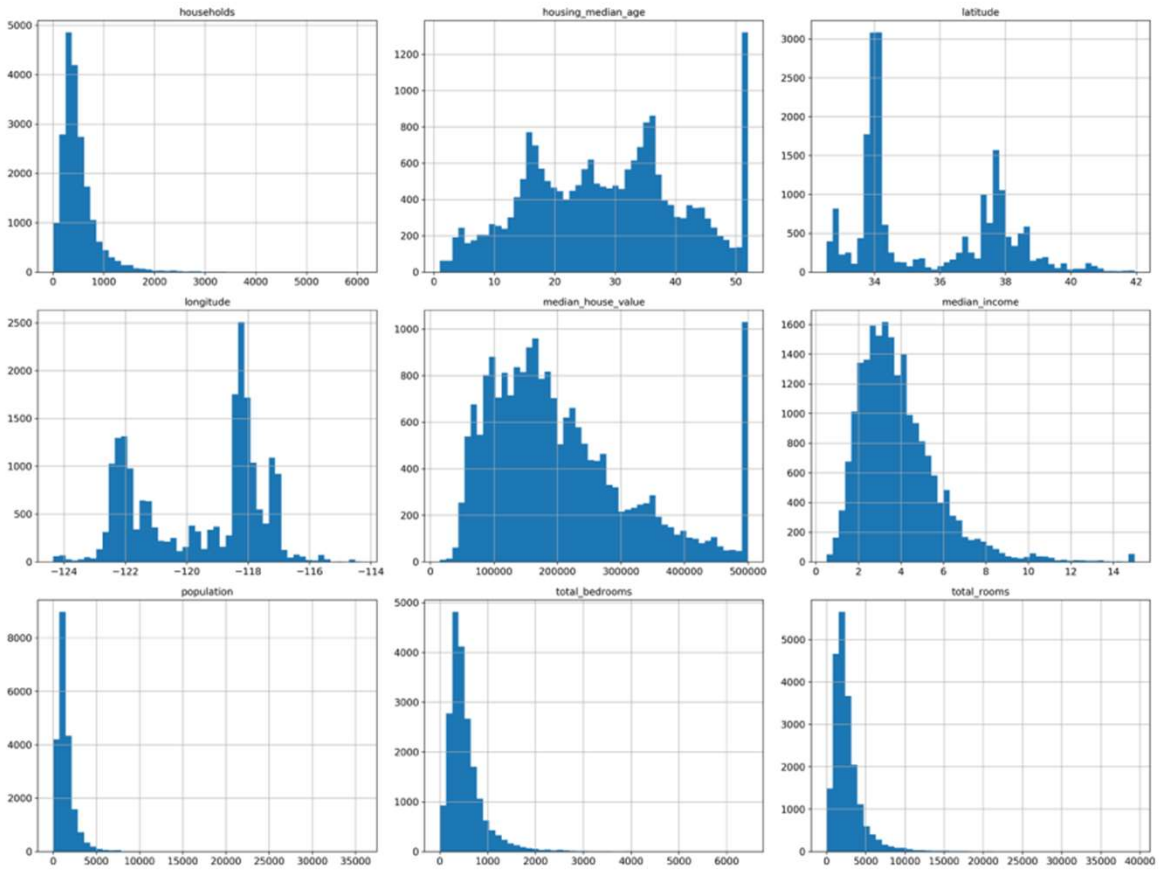
$$\mathrm{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m}\sum_{i=1}^{m}\left(h\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right)^2} \qquad \mathrm{MAE}(\mathbf{X}, h) = \frac{1}{m}\sum_{i=1}^{m}\left|h\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right|$$

RMSE is more sensitive to outliers than MAE;

If outliers are exponentially rare, RMSE is the preferred metric

ONCE SELECTED, THE PERFORMANCE METRIC MUST NOT CHANGE FOR THE ENTIRE MODEL TRAINING PROCESS!!!

The Data: Quick Look



Observations:

Median income data was scaled and capped;

Housing median age and median house value are also capped:

- Remove the capped data or
- Collect more data

The attributes have different scales:

- Feature scaling must be applied

Many histograms are tail-heavy:

- Should be transformed into bell-shaped distribution

Creating a Test Set

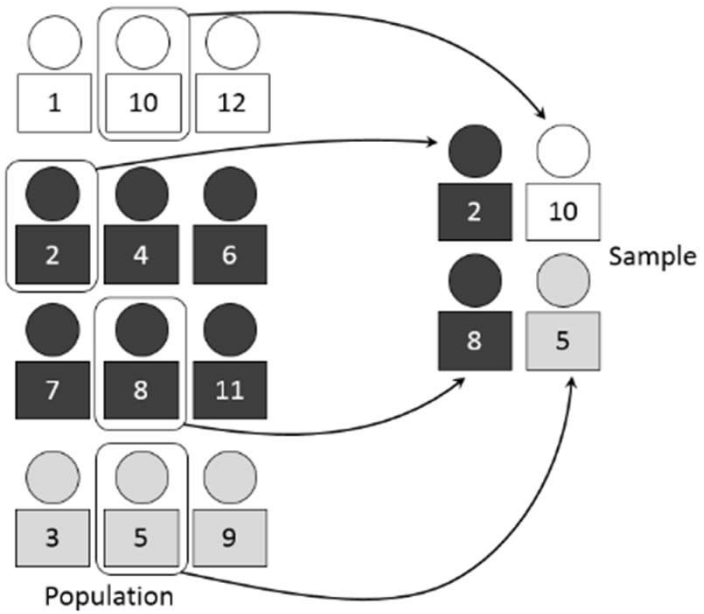
- Test set is used to test performance of the ML model
- Data in the test set must not be used in training the model!
  - To avoid the **data snooping** bias
  - Student exam analogy
    - If the students are tested on exact same questions they saw in the learning – it does not test knowledge in the subject
    - A student must be able to generalize the learned knowledge on new tasks
  - A well-trained ML model must generalize well

Creating a Test Set

- Randomly picking the test set instances (typically 20%) is not good
  - Repeated generation of the test set results into the algorithm observing the entire dataset
- Solutions (not perfect)
  - Set aside the test set on the first run and load it in subsequent runs
  - Use same random number generator's seed value
- Will not work if the dataset is updated
- Perfect solution
  - Identifier-based split: use hash

Creating a Test Set: Stratified Sampling

- The test set must be representative of the entire dataset
  - All categories of data in the test dataset must have the same proportions as in the entire dataset
  - The categories are defined by the most important attribute (feature) of the data



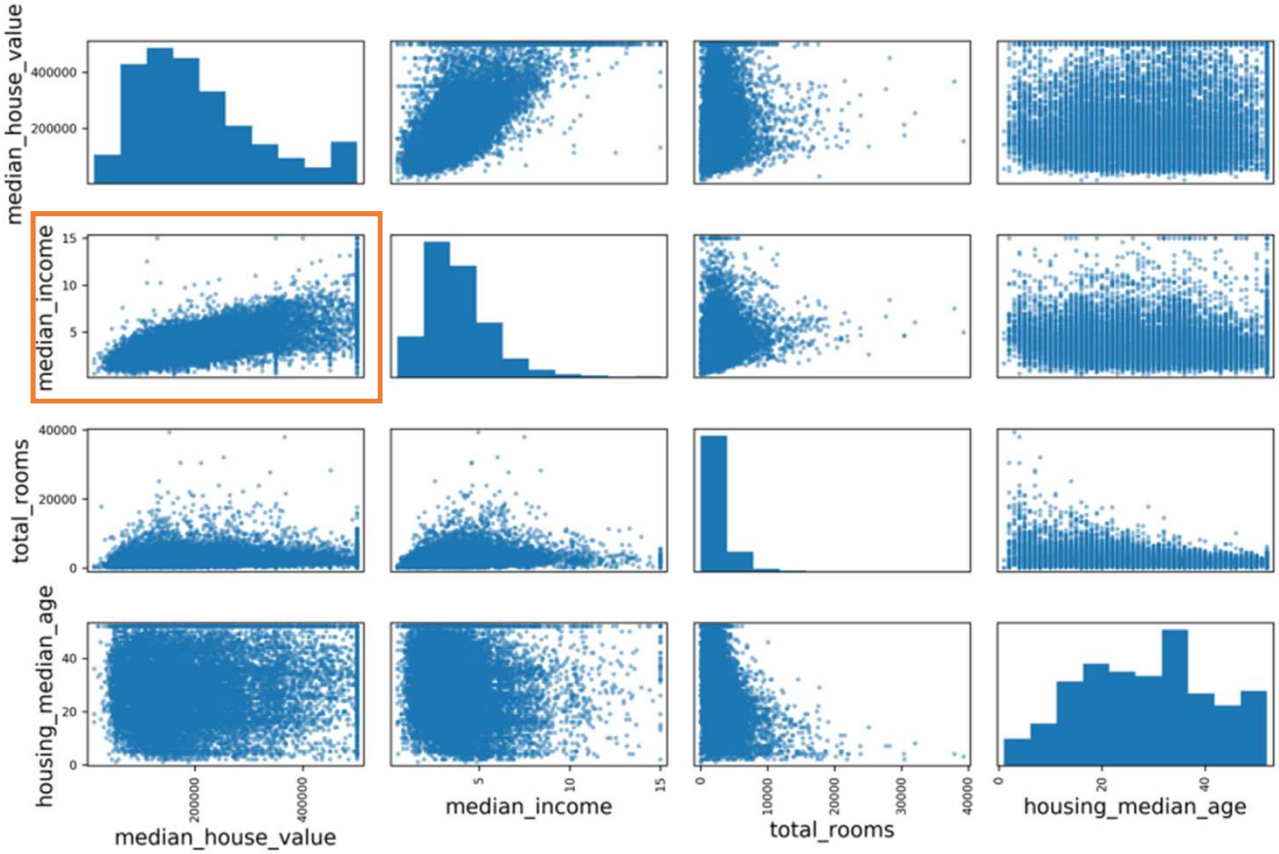
Feature Engineering

The process of modifying the data with objective to optimize training of the ML model

Feature Engineering

- Exploratory data analysis
- Feature creation
- Transformation
- Feature extraction

Feature Correlations



Finding important features:

Median income is well correlated with the median house values

Combining Features Together

- Example:
  - Rooms per household
  - Bedrooms per room
  - Population per household

median_house_value	1.000000
median_income	0.687160
rooms_per_household	0.146285
total_rooms	0.135097
housing_median_age	0.114110
households	0.064506
total_bedrooms	0.047689
population_per_household	-0.021985
population	-0.026920
longitude	-0.047432
latitude	-0.142724
bedrooms_per_room	-0.259984

Handling Text and Categorical Attributes

- The ocean_proximity attribute takes four values
  - <1H OCEAN
  - INLAND
  - ISLAND
  - NEAR BAY
  - NEAR OCEAN
- Solutions:
  - Convert it to a numerical attribute with values [0,1,2,3,4]
    - Introduces unnecessary numerical relations between the values
  - One-hot encoding

One-Hot Encoding

Color
Red
Red
Yellow
Green
Yellow

Red	Yellow	Green
1	0	0
1	0	0
0	1	0
0	0	1

https://www.tutorialexample.com/encode-word-n-grams-to-one-hot-encoding-with-numpy-deep-learning-tutorial/

Feature Scaling

- ML algorithms perform best when the features have similar scales
- The housing dataset
  - Total number of room is between 6 and 39 320
  - The median incomes range from 0 to 15

- Two common methods
  - Min-max scaling
    - All values are converted to the [0,1] range
  - Standardization
    - For each numerical feature: Subtract the mean value and divide by the standard deviation

Data Cleaning

The process of removing errors and inconsistencies from the data

Data Quality

- Validity
- Accuracy
- Completeness
- Consistency
- Uniformity

- Validity
  - Data-type constraints
  - Range constraints
  - Mandatory constraints
  - Unique constraints
  - Set-membership constraints
  - Foreign-Key constraints
  - Regular expression patterns
  - Cross-field validation

Missing Features

- Delete the corresponding data instances
- Delete the entire attribute (feature)
- Substitute the missing values with
  - Zero, the mean, the median
  - Task-specific

Select and Train a Model

A Linear Regression Model

- RMSE = $68,628
  - With median_housing_values between $120,000 and $265,000 this is too much of an error
  - Apparently, the model is underfitting the training data
  - Two possible reasons
    - The features do not provide enough information to make good predictions
    - The model is not powerful enough
  - Possible solutions
    - Select a more powerful model
    - Add better features to the data
    - Add constraints on the model

A Decision Tree Regressor

- RMSE = 0 (!!!)
  - The model badly overfit the data
  - If you check the code, the predictions are made on data instances from the training set

K-Fold Cross Validation

https://towardsdatascience.com/cross-validation-k-fold-vs-monte-carlo-e54df2fc179b

The Decision Tree Regressor with 10-fold CV

```
>>> display_scores(tree_rmse_scores)
Scores: [70194.33680785 66855.16363941 72432.58244769 70758.73896782
 71115.88230639 75585.14172901 70262.86139133 70273.6325285
 75366.87952553 71231.65726027]
Mean: 71407.68766037929
Standard deviation: 2439.4345041191004
```

A Linear Regressor with 10-fold CV

```
>>> display_scores(lin_rmse_scores)
Scores: [66782.73843989 66960.118071   70347.95244419 74739.57052552
 68031.13388938 71193.84183426 64969.63056405 68281.61137997
 71552.91566558 67665.10082067]
Mean: 69052.46136345083
Standard deviation: 2731.674001798348
```

A Random Forest Regressor with 10-fold CV

```
>>> forest_rmse
18603.515021376355
```

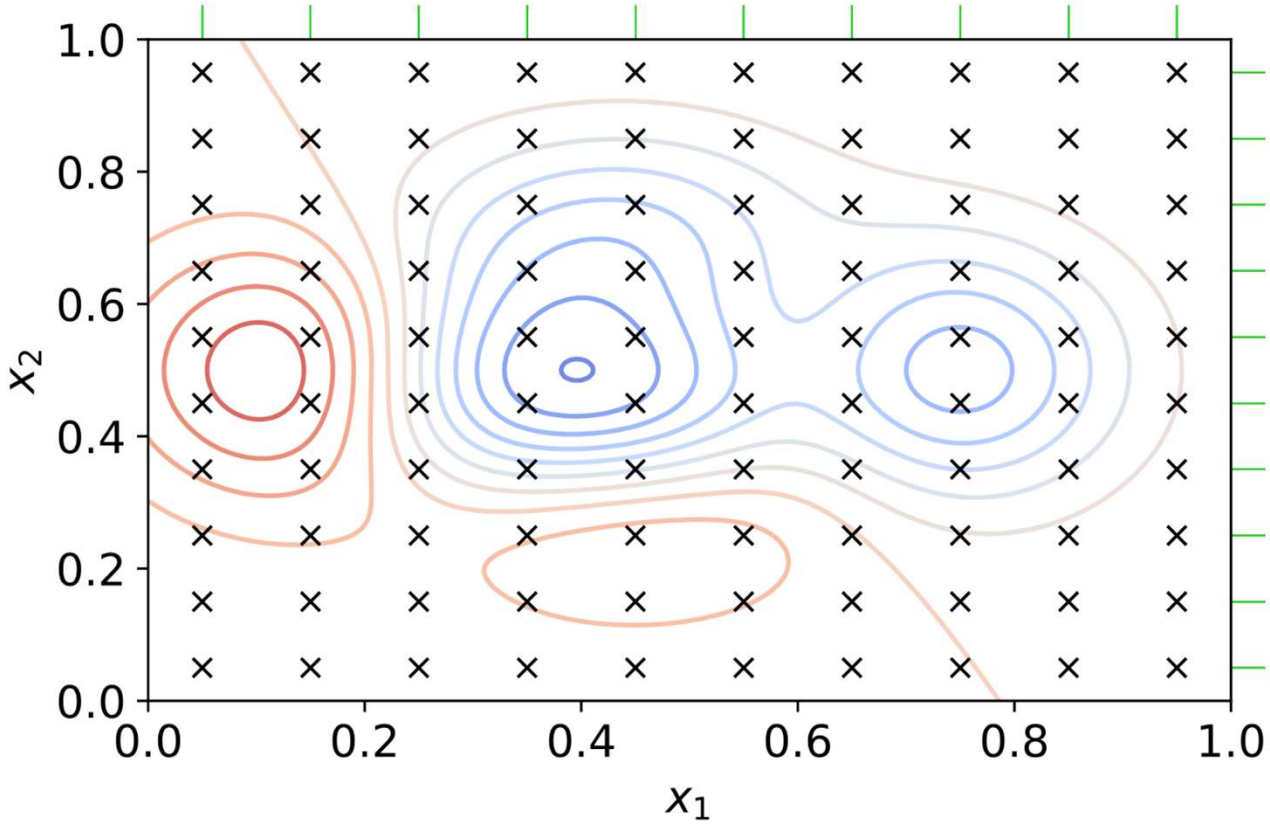
```
>>> display_scores(forest_rmse_scores)
Scores: [49519.80364233 47461.9115823  50029.02762854 52325.28068953
 49308.39426421 53446.37892622 48634.8036574  47585.73832311
 53490.10699751 50021.5852922 ]
Mean: 50182.303100336096
Standard deviation: 2097.0810550985693
```

- This is better but RMSE is still too high: the model is overfitting
- Solutions
  - Simplify the model
  - Introduce constraints
  - Get more training data

- In practice many more ML algorithms should be tried before choosing the one to tweak.

Fine-Tuning the Model

Grid Search



https://en.wikipedia.org/wiki/Hyperparameter_optimization

Grid Search

```
from sklearn.model_selection import GridSearchCV

param_grid = [
    {'n_estimators': [3, 10, 30], 'max_features': [2, 4, 6, 8]},
    {'bootstrap': [False], 'n_estimators': [3, 10], 'max_features': [2, 3,
4]},
  ]

forest_reg = RandomForestRegressor()

grid_search = GridSearchCV(forest_reg, param_grid, cv=5,
                           scoring='neg_mean_squared_error',
                           return_train_score=True)

grid_search.fit(housing_prepared, housing_labels)
```

```
>>> grid_search.best_params_
{'max_features': 8, 'n_estimators': 30}
```

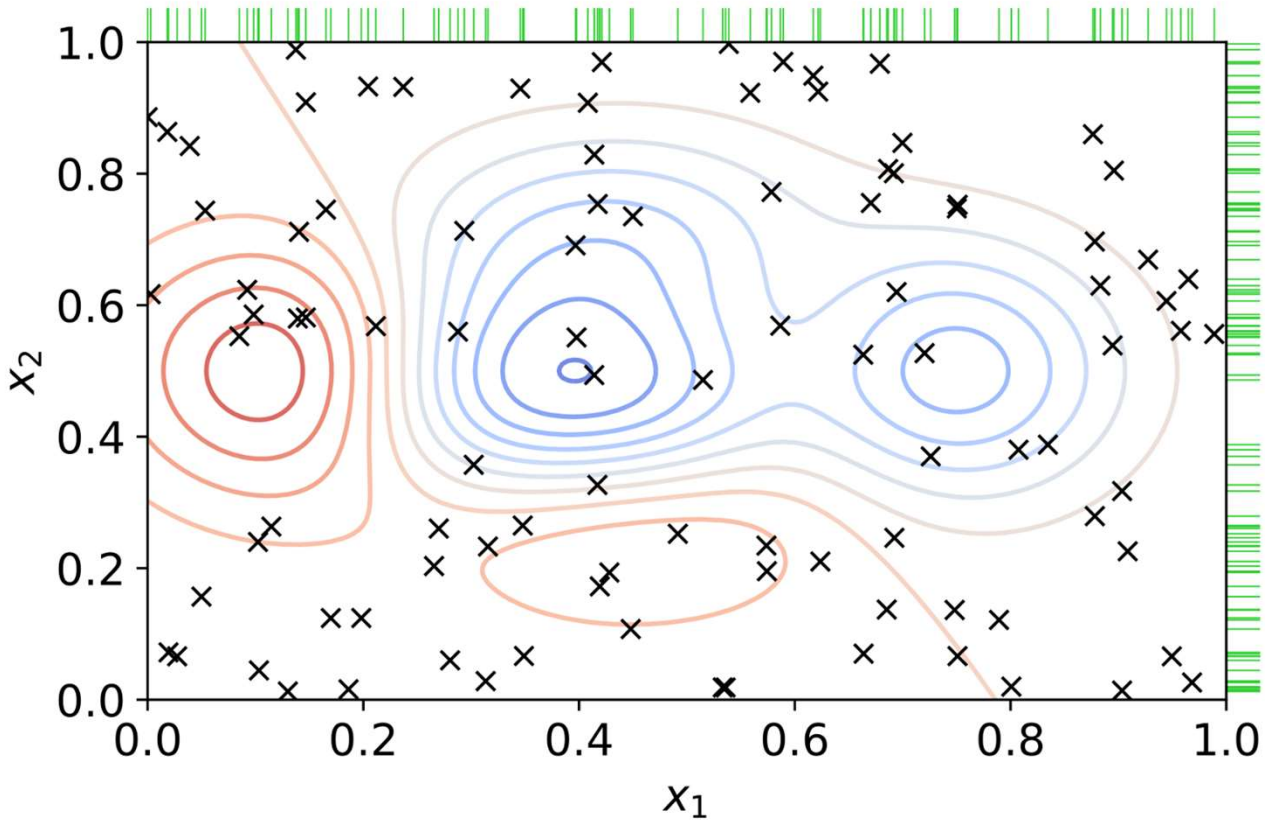
Grid Search

```
>>> grid_search.best_estimator_
RandomForestRegressor(bootstrap=True, criterion='mse', max_depth=None,
           max_features=8, max_leaf_nodes=None, min_impurity_decrease=0.0,
           min_impurity_split=None, min_samples_leaf=1,
           min_samples_split=2, min_weight_fraction_leaf=0.0,
           n_estimators=30, n_jobs=None, oob_score=False, random_state=None,
           verbose=0, warm_start=False)
```

Grid Search

```
63669.05791727153 {'max_features': 2, 'n_estimators': 3}
55627.16171305252 {'max_features': 2, 'n_estimators': 10}
53384.57867637289 {'max_features': 2, 'n_estimators': 30}
60965.99185930139 {'max_features': 4, 'n_estimators': 3}
52740.98248528835 {'max_features': 4, 'n_estimators': 10}
50377.344409590376 {'max_features': 4, 'n_estimators': 30}
58663.84733372485 {'max_features': 6, 'n_estimators': 3}
52006.15355973719 {'max_features': 6, 'n_estimators': 10}
50146.465964159885 {'max_features': 6, 'n_estimators': 30}
57869.25504027614 {'max_features': 8, 'n_estimators': 3}
51711.09443660957 {'max_features': 8, 'n_estimators': 10}
49682.25345942335 {'max_features': 8, 'n_estimators': 30}
62895.088889905004 {'bootstrap': False, 'max_features': 2, 'n_estimators': 3}
54658.14484390074 {'bootstrap': False, 'max_features': 2, 'n_estimators': 10}
59470.399594730654 {'bootstrap': False, 'max_features': 3, 'n_estimators': 3}
52725.01091081235 {'bootstrap': False, 'max_features': 3, 'n_estimators': 10}
57490.612956065226 {'bootstrap': False, 'max_features': 4, 'n_estimators': 3}
51009.51445842374 {'bootstrap': False, 'max_features': 4, 'n_estimators': 10}
```

Randomized Search



https://en.wikipedia.org/wiki/Hyperparameter_optimization